SIGNAL PIPELINE PROCESS USING ROS2

ABSTRACT

Abstract:

The **ROS 2 Signal Processing Pipeline** is a modular software system developed to demonstrate real-time signal acquisition, processing, and analysis using the Robot Operating System (ROS 2). The project combines modern C++ programming, Python integration, reusable digital signal processing (DSP) algorithms, and distributed communication through ROS 2 nodes.

The implementation consists of four primary components: a standalone C++ DSP library, Python bindings created using pybind11, a ROS 2 publisher node responsible for generating synthetic signals or replaying recorded sensor data, and dedicated C++ and Python processing nodes that subscribe to published signals and apply filtering operations.

The standalone DSP library provides reusable implementations of common filtering techniques, including Moving Average, Median Filter, Low Pass Filter, and supporting data structures such as a Ring Buffer. Through pybind11, the same C++ filtering functionality is exposed to Python, allowing identical algorithms to be executed in both languages while maintaining consistency and minimizing duplicate code.

The ROS 2 publisher supports multiple operating modes, including synthetic signal generation and replay from CSV data. Configurable options such as Gaussian noise, signal jitter, and packet drop simulation enable realistic testing of the signal processing pipeline.

The project demonstrates interoperability between C++ and Python, modular software architecture, efficient data processing, and modern software engineering practices including version control with Git, build automation using CMake and Colcon, and comprehensive validation through ROS 2 topics and node communication.

Overall, the implementation provides a flexible foundation for real-time signal processing applications and serves as an effective demonstration of ROS 2-based software development.

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

The rapid advancement of robotics, embedded systems, and autonomous technologies has increased the demand for efficient and reliable real-time signal processing solutions. Modern robotic systems rely on continuous streams of sensor data to make accurate decisions, requiring software capable of processing, filtering, and analysing signals with minimal latency. The Robot Operating System 2 (ROS 2) provides a distributed communication framework that enables modular software development through publishers, subscribers, services, and actions, making it well suited for such applications.

The objective of this project is to design and implement a **ROS 2 Signal Processing Pipeline** using a combination of **C++17**, **Python 3**, and **pybind11**. The system demonstrates how reusable Digital Signal Processing (DSP) algorithms can be integrated into ROS 2 applications while maintaining interoperability between C++ and Python. A standalone C++ DSP library was developed to implement common filtering algorithms, including Moving Average, Median Filter, Low Pass Filter, and Ring Buffer data structures. These algorithms were then exposed to Python using pybind11, allowing the same C++ implementation to be accessed from Python without duplicating logic.

The project is organized into multiple ROS 2 packages, each responsible for a specific task. A publisher node generates synthetic sensor data or replays previously recorded measurements from a CSV file. Separate C++ and Python processing nodes subscribe to the published topics, apply filtering operations, and display processed results. This modular design demonstrates the flexibility of ROS 2 while promoting code reuse and maintainability.

Throughout the implementation, modern software engineering practices such as modular architecture, version control using Git, build automation with CMake and Colcon, and comprehensive validation procedures were followed. The project therefore serves as a practical demonstration of developing scalable and reusable robotics software using ROS 2.

CHAPTER 2

PROBLEM STATEMENT

Real-time robotic applications often receive noisy and inconsistent sensor data from multiple hardware devices. Directly processing this raw data can reduce system accuracy and reliability, leading to unstable control decisions or incorrect environmental perception. Therefore, sensor data must be filtered before being used by higher-level algorithms.

Many existing implementations tightly couple filtering algorithms with application logic, making them difficult to reuse across different programming languages or robotic systems. Additionally, implementing identical algorithms separately in C++ and Python increases maintenance effort and introduces the possibility of inconsistent behavior.

The challenge addressed by this project is to design a reusable signal processing framework capable of:

- Processing sensor data in real time.
- Supporting both C++ and Python applications.
- Providing reusable Digital Signal Processing filters.
- Publishing and subscribing to ROS 2 topics efficiently.
- Simulating realistic sensor conditions through noise, jitter, and replay functionality.
- Maintaining modularity, scalability, and maintainability.

The developed solution addresses these challenges by creating a standalone C++ filtering library integrated with ROS 2 and exposing the same functionality to Python using pybind11.

CHAPTER 3

OBJECTIVES

The primary objective of this project is to develop a modular and reusable signal processing framework based on ROS 2 that demonstrates interoperability between C++ and Python while implementing common Digital Signal Processing techniques.

The specific objectives include:

- Design and implement a standalone C++ DSP library.
- Develop reusable filtering algorithms including:
    - Ring Buffer
    - Moving Average Filter
    - Median Filter
    - Low Pass Filter
- Support fixed-point signal processing using Q15 arithmetic where applicable.
- Create Python bindings for the complete C++ library using pybind11.
- Develop a ROS 2 publisher capable of:
    - Synthetic signal generation
    - Replaying CSV sensor data
    - Simulating Gaussian noise
    - Simulating signal jitter
    - Simulating packet loss
- Develop a C++ ROS 2 subscriber node for signal processing.
- Develop a Python ROS 2 subscriber node using the C++ DSP library through pybind11.
- Validate communication between publisher and subscriber nodes.
- Verify consistent filtering results between C++ and Python implementations.
- Manage the complete project using Git and GitHub.

CHAPTER 4

SCOPE OF THE PROJECT

The scope of this project is limited to the implementation and validation of a modular signal processing framework within the ROS 2 ecosystem. The project focuses on software architecture, signal filtering, and cross-language interoperability rather than direct interaction with physical hardware.

The implemented system is capable of generating synthetic sensor signals, replaying recorded data from CSV files, and processing those signals using reusable DSP algorithms. The standalone C++ library serves as the core processing engine, while Python bindings provide identical functionality to Python-based ROS 2 nodes. This architecture demonstrates how performance-critical algorithms can remain in C++ while allowing higher-level application development in Python.

Although the project simulates realistic sensor behavior through configurable noise, jitter, and packet drop mechanisms, it does not include direct integration with physical sensors or embedded hardware. Nevertheless, the modular design allows such components to be incorporated in future work with minimal modifications.

The project is intended for educational purposes, software engineering assessments, and as a foundation for future robotics, IoT, and autonomous system applications.

CHAPTER 5

SYSTEM REQUIREMENTS

5.1 Hardware Requirements

The minimum hardware configuration required for successful execution of the project is shown below.

Component	**Specification**
Processor	Intel Core i5 (or equivalent)
RAM	Minimum 8 GB
Storage	20 GB free disk space
Operating System	Ubuntu 24.04 LTS
Internet	Required for GitHub and ROS package installation

5.2 Software Requirements

Software	**Version**
Ubuntu Linux	24.04 LTS
ROS 2	Jazzy
Python	3.x
GCC / G++	C++17 Compatible
CMake	3.10 or later
Colcon	Latest
pybind11	Latest Stable Version
Git	Latest
GitHub	Remote Repository
Visual Studio Code	Latest (Optional)

5.3 Libraries Used

The project uses several open-source libraries and frameworks to implement the required functionality:

- ROS 2 Jazzy for distributed communication.
- Standard Template Library (STL) for C++ containers and algorithms.
- pybind11 for exposing C++ classes to Python.
- CMake for build configuration.
- Colcon for ROS 2 workspace compilation.
- Python standard libraries for CSV handling and signal replay.
- Git for version control and project management.

.

CHAPTER 6

TECHNOLOGIES

The project integrates several modern software technologies to develop a modular and reusable signal processing pipeline.

Technology	Purpose
ROS 2 Jazzy	Communication framework between publisher and subscriber nodes
C++17	Development of the standalone DSP library and C++ processing node
Python 3	Publisher implementation, Python processing node, testing
pybind11	Exposing C++ DSP library to Python
CMake	Building C++ libraries
Colcon	Building the ROS 2 workspace
Git & GitHub	Version control and source code management
CSV	Storage and replay of sensor data

USESES:

ROS 2 provides a scalable middleware for modular robotics software.

C++17 offers high performance for signal processing.

Python simplifies application logic and testing.

pybind11 enables reuse of C++ algorithms in Python without duplicating code.

Git tracks source code changes and supports collaborative development.

CHAPTER 7

PROJECT ARCHITECTURE

The project follows a layered and modular architecture. Each component has a clearly defined responsibility, improving maintainability and allowing independent testing.

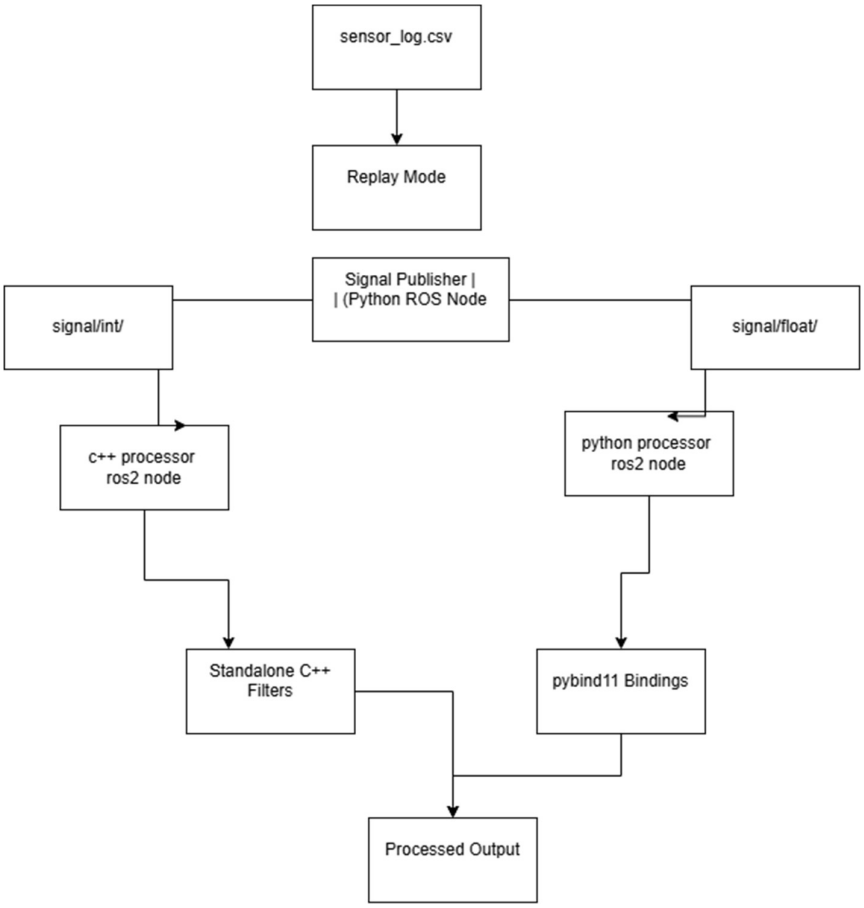


Fig : ARCHITETURE OF ROS2

Data Flow

1. Publisher generates synthetic or replayed data.
2. Messages are published on ROS 2 topics.
3. Both processing nodes subscribe to the topics.
4. The C++ node directly uses the standalone DSP library.
5. The Python node accesses the same library through pybind11.
6. Filtered results are displayed.

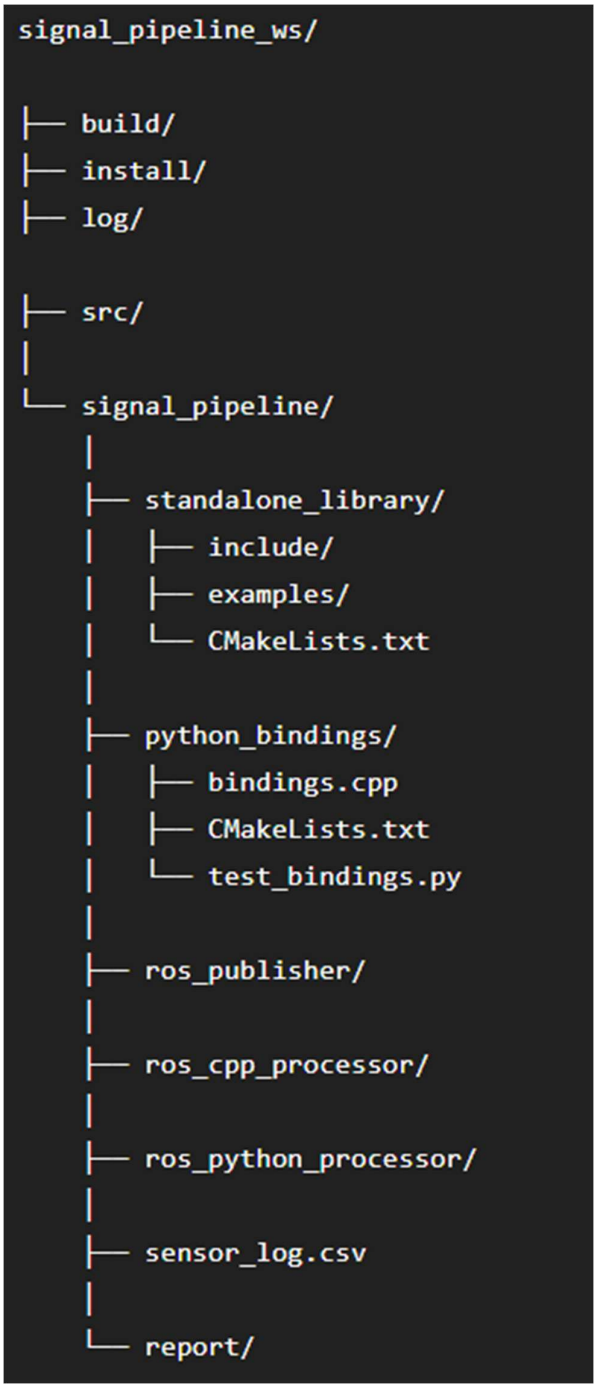
This design avoids duplicate filter implementations and ensures identical processing logic across languages.

CHAPTER 8

PROJECT DIRECTORY STRUCTURE

Project Directory Structure

The project is organized into independent modules.



Folder Description:

Folder	Description
standalone_library	C++ DSP filters
python_bindings	pybind11 wrapper
ros_publisher	Signal publisher node
ros_cpp_processor	C++ processing node
ros_python_processor	Python processing node
report	Documentation
sensor_log.csv	Replay dataset

CHAPTER 9

STANDALONE C++ DSP LIBRARY

The standalone DSP library is the core component of the project. It contains reusable filtering algorithms implemented using modern C++.

The library was intentionally developed independently of ROS 2 so that it can be reused in embedded systems, desktop applications, or future robotics projects.

Main Components

1. Ring Buffer
2. Filter Base
3. Moving Average Filter
4. Median Filter
5. Low Pass Filter

Ring Buffer:

Code:

```
#ifndef RINGBUFFER_HPP

#define RINGBUFFER_HPP

#include <array>

#include <cstddef>

template<typename T, std::size_t Size>

class RingBuffer

{

public:

RingBuffer()

      : head_(0),

        count_(0)

  {

  }

 void push(const T& value)

  {

     buffer_[head_] = value;
```

```
  head_ = (head_ + 1) % Size;
if(count_ < Size)
          count_++;
   }
  T get(std::size_t index) const
   {
      std::size_t pos =
          (head_ + Size - count_ + index) % Size;
return buffer_[pos];
   }
std::size_t size() const
   {
      return count_;
   }
 bool full() const
   {
      return count_ == Size;
   }
private:
std::array<T, Size> buffer_;
   std::size_t head_;
   std::size_t count_;
};
#endif
```

Filter Base

Code:

```
#ifndef FILTERBASE_HPP
#define FILTERBASE_HPP
template<typename T>
class FilterBase
{
public:
virtual T update(T sample, double dt) =0;
virtual void reset()=0;
  virtual ~FilterBase()=default;
};
#endif
```

Moving Average

Code:

```
#ifndef MOVINGAVERAGE_HPP
#define MOVINGAVERAGE_HPP
#include "RingBuffer.hpp"
#include "FilterBase.hpp"
#include <type_traits>
#include <cstdint>
template<typename T, std::size_t WindowSize>
class MovingAverage : public FilterBase<T>
{
private:
    using Accumulator = std::conditional_t<
        std::is_integral_v<T>,
        int64_t,
        double>;
```

```
  RingBuffer<T, WindowSize> buffer_;
    Accumulator sum_;
    bool filled_;
public:
    MovingAverage()
        : sum_(0),
          filled_(false)
    {
    }
T update(T sample, double dt) override
    {
        (void)dt;
// Remove oldest sample when buffer is full
        if (buffer_.full())
        {
            sum_ -= static_cast<Accumulator>(buffer_.get(0));
        }
// Insert newest sample
        buffer_.push(sample);
  // Add newest sample to running sum
        sum_ += static_cast<Accumulator>(sample);
  // Update buffer status
        filled_ = buffer_.full();
// Current number of valid samples
        const std::size_t count = buffer_.size();
        if (count == 0)
        {
            return T{};
        }   if constexpr (std::is_integral_v<T>)
```

```
        {
            return static_cast<T>(
                sum_ / static_cast<Accumulator>(count));
        }
        else
        {
            return static_cast<T>(
                sum_ / static_cast<double>(count));
        }
    }

    void reset() override
    {
        sum_ = 0;
        filled_ = false;
        buffer_ = RingBuffer<T, WindowSize>();
    }
bool isFilled() const
    {
        return filled_;
    }
};
#endif
```

Lowpass Filter

The Low Pass Filter attenuates high-frequency components while preserving slowly changing signals.

Equation

$y(n) = \alpha x(n) + (1 - \alpha) y(n - 1)$ y(n)=αx(n)+(1−α)y(n−1) where

α = smoothing factor

x = current sample

y = filtered output

Code:

```
#ifndef LOWPASSFILTER_HPP
#define LOWPASSFILTER_HPP
#include <cstdint>
#include <cmath>
namespace signal_pipeline
{
class LowPassFilter
{
public:
   explicit LowPassFilter(float cutoff_frequency_hz = 1.0f)
      : cutoff_frequency_(cutoff_frequency_hz),
        previous_output_(0.0f),
        initialized_(false)
   {
   }
   void setCutoffFrequency(float cutoff_frequency_hz)
   {
      cutoff_frequency_ = cutoff_frequency_hz;
   }
```

```
 float getCutoffFrequency() const
    {
        return cutoff_frequency_;
    }
void reset(float value = 0.0f)
    {
        previous_output_ = value;
        initialized_ = false;
    }
float update(float input, float dt)
    {
        if (dt <= 0.0f)
            return previous_output_;
if (!initialized_)
        {
            previous_output_ = input;
            initialized_ = true;
            return previous_output_;
        }
            constexpr float PI = 3.14159265358979323846f;
            float rc = 1.0f / (2.0f * PI * cutoff_frequency_);
            float alpha = dt / (rc + dt);
            previous_output_ += alpha * (input - previous_output_);
            return previous_output_;
    }
private:
    float cutoff_frequency_;
    float previous_output_;
    bool initialized_;
```

```
};
class LowPassFilterQ15
{
public:
    explicit LowPassFilterQ15(uint16_t alpha_q15 = 16384)
        : alpha_q15_(alpha_q15),
          previous_output_(0),
          initialized_(false)
    {
    }
void setAlphaQ15(uint16_t alpha_q15)
    {
        alpha_q15_ = alpha_q15;
    }
    uint16_t getAlphaQ15() const
    {
        return alpha_q15_;
    }
    void computeAlpha(float cutoff_frequency_hz, float dt)
    {
        constexpr float PI = 3.14159265358979323846f;
        float rc = 1.0f / (2.0f * PI * cutoff_frequency_hz);
        float alpha = dt / (rc + dt);
        if (alpha < 0.0f)
            alpha = 0.0f;
        if (alpha > 1.0f)
            alpha = 1.0f;
        alpha_q15_ = static_cast<uint16_t>(alpha * 32768.0f);
    }
```

```
void reset(int32_t value = 0)
    {
        previous_output_ = value;
        initialized_ = false;
    }
int32_t update(int32_t input)
    {
        if (!initialized_)
        {
            previous_output_ = input;
            initialized_ = true;
            return previous_output_;
        }
int32_t error = input - previous_output_;
         previous_output_ +=
            static_cast<int32_t>(
                (static_cast<int64_t>(alpha_q15_) * error) >> 15);
        return previous_output_;
    }
private:
    uint16_t alpha_q15_;
    int32_t previous_output_;
    bool initialized_;
};
} // namespace signal_pipeline

#endif // LOWPASSFILTER_HPP
```

Median Filter

Code:

```
#ifndef MEDIANFILTER_HPP
#define MEDIANFILTER_HPP
#include <array>
#include <cstddef>
#include <cstdint>
namespace signal_pipeline
{
template<typename T, std::size_t WindowSize>
class MedianFilter
{
   static_assert(WindowSize > 0,
                 "WindowSize must be greater than zero.");
public:
   MedianFilter()
      : index_(0),
        count_(0)
   {
      buffer_.fill(T{});
   }
   void reset()
   {
      buffer_.fill(T{});
      index_ = 0;
      count_ = 0;
   }
T update(T sample)
   {
```

```
// Store newest sample into ring buffer
buffer_[index_] = sample;
index_ = (index_ + 1) % WindowSize;
if (count_ < WindowSize)
   ++count_;
// Copy only valid samples
std::array<T, WindowSize> temp{};
for (std::size_t i = 0; i < count_; ++i)
{
   temp[i] = buffer_[i];
}
// Insertion Sort
for (std::size_t i = 1; i < count_; ++i)
{
   T key = temp[i];
   std::size_t j = i;
   while (j > 0 && temp[j - 1] > key)
   {
      temp[j] = temp[j - 1];
      --j;
   }
   temp[j] = key;
}
// Return median
if (count_ % 2 == 1)
{
   return temp[count_ / 2];
}
else
```

```
        {
            return static_cast<T>(
                (temp[count_ / 2 - 1] + temp[count_ / 2]) / 2);
        }
    }
constexpr std::size_t windowSize() const
    {
        return WindowSize;
    }

    std::size_t sampleCount() const
    {
        return count_;
    }

    bool isFull() const
    {
        return count_ == WindowSize;
    }
private:
std::array<T, WindowSize> buffer_;

    std::size_t index_;

    std::size_t count_;
};
} // namespace signal_pipeline

#endif // MEDIANFILTER_HPP
```

CHAPTER 11

ROS 2 SIGNAL PUBLISHER

11.1 Overview

The Signal Publisher is the data source of the signal processing pipeline. It is implemented as a ROS 2 Python node and is responsible for generating or replaying sensor data and publishing it to ROS 2 topics.

The node supports two operating modes:

Synthetic Mode – Generates artificial signals such as sine waves with optional Gaussian noise.

Replay Mode – Reads previously recorded sensor values from sensor_log.csv and republishes them.

This design enables testing with both simulated and recorded data without changing the processing nodes.

11.2 Features

The publisher supports the following functionality:

Synthetic signal generation

Replay from CSV file

Gaussian noise simulation

Signal jitter

Packet drop simulation

Integer and floating-point message publishing

Configurable publishing rate

11.3 ROS 2 Topics

Topic	Message Type	Purpose
/signal/int	std_msgs/msg/Int32	Integer signal
/signal/float	std_msgs/msg/Float32	Floating-point signal

11.4 Publisher Initialization:

from std_msgs.msg import Int32

from std_msgs.msg import Float32

self.int_pub = self.create_publisher(Int32,"/signal/int",10)

self.float_pub = self.create_publisher(Float32,"/signal/float",10)

11.5 Publishing Loop

During execution the node periodically publishes values.

int_msg = Int32()

float_msg = Float32()

int_msg.data = int(value)

float_msg.data = float(value)

self.int_pub.publish(int_msg)

self.float_pub.publish(float_msg)

11.6 Synthetic Mode

Synthetic Mode generates mathematical waveforms for testing.

Typical characteristics include:

- Sine wave generation
- Multiple frequencies
- Gaussian noise
- Random signal variation

Illustrative formula:

$$x(t) = A\sin(2\pi ft)$$

where:

- A= amplitude
- f= frequency

This mode is useful when physical sensors are unavailable.

11.7 Replay Mode

Replay Mode reads values from:

sensor_log.csv

CHAPTER 12

ROS 2 C++ PROCESSOR

12.1 Overview

The C++ Processor subscribes to the publisher topics and performs digital filtering using the standalone DSP library.

Unlike the publisher, this node focuses entirely on processing incoming data.

12.2 Responsibilities

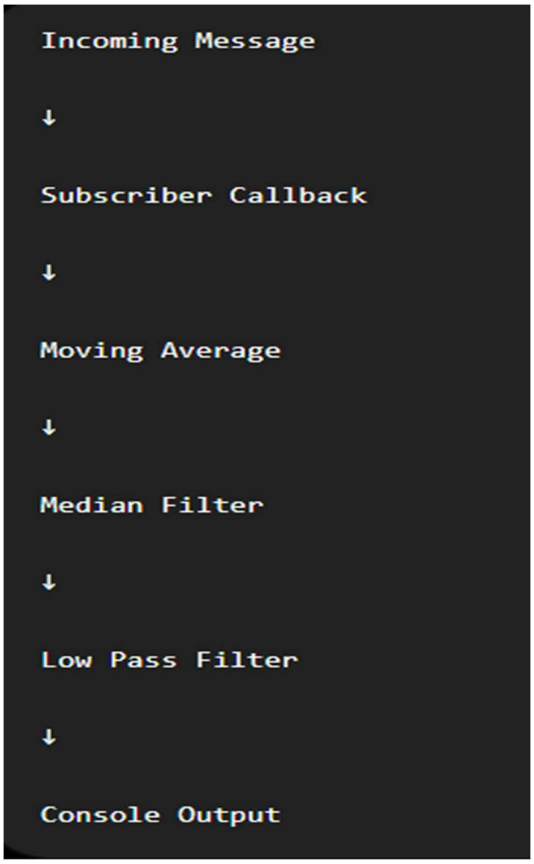
The node performs:

Topic subscription

Filter execution

Processed output display

12.3 Processing Flow



CHAPTER 13

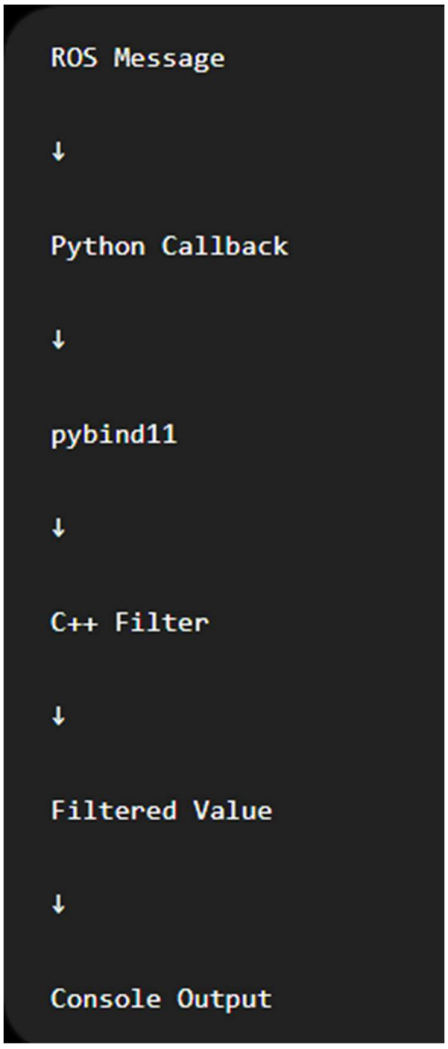
ROS 2 PYTHON PROCESSOR

13.1 Overview

The Python Processor demonstrates interoperability between ROS 2 Python applications and the standalone C++ DSP library.

Instead of implementing filters again in Python, the node directly uses the pybind11 bindings.

13.2 Workflow :



13.3 Importing the Library

```
import signal_pipeline
```

13.4 Creating Filters

```
filter = signal_pipeline.LowPassFilter(0.2)
```

13.5 Subscriber Callback

def callback(msg):

filtered = filter.update(msg.data)

print(filtered)

The callback executes every time the publisher transmits a message.

13.6 Advantages :

Using pybind11 provides several benefits:

No duplicate DSP implementation

Faster execution than pure Python

Shared C++ codebase

Easier maintenance

13.7 Comparison

C++ Processor	Python Processor
Native C++	Python
Uses DSP Library	Uses pybind11
High Performance	Easy Development
Direct Compilation	Interpreted Execution

CHAPTER 14

SIGNAL PROCESSING WORKFLOW

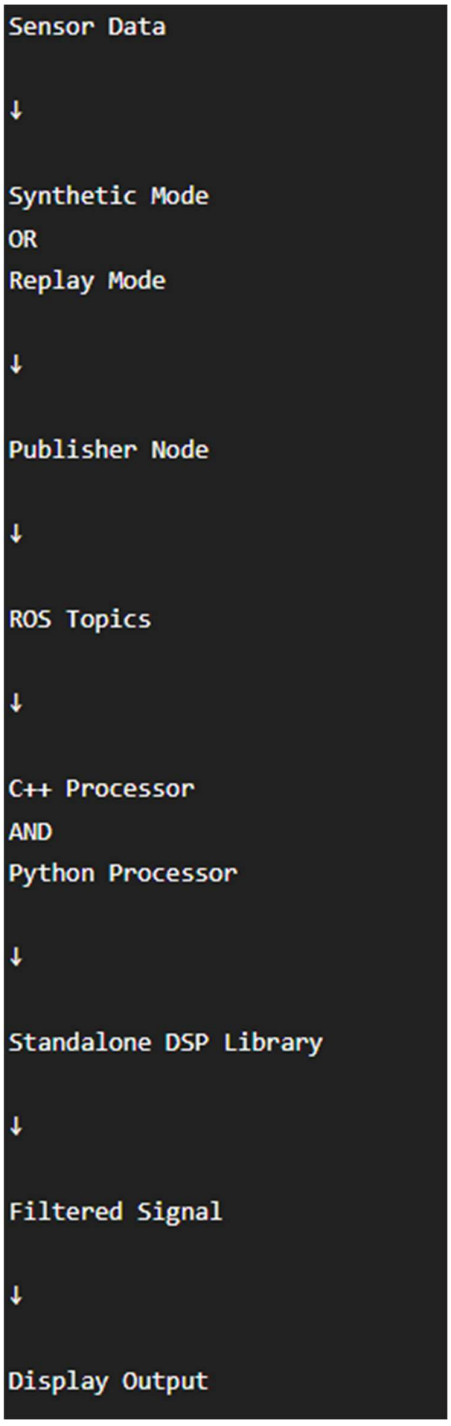


Fig: **Signal Processing Workflow**

Processing Steps

1. Publisher initializes.
2. Signal source selected.
3. Signal generated or replayed.
4. Message published.
5. Subscriber receives message.
6. Filter processes sample.
7. Result displayed.

CHAPTER 15

BUILD AND EXECUTION PROCEDURE

The project was compiled using Colcon.

Step 1 Open the workspace.

Cd ~/signal_pipeline_ws

Step 2 Source ROS 2.

source /opt/ros/jazzy/setup.bash

Step 3 Build.

colcon build

Step 4

Source the workspace.

source install/setup.bash

Running the Project

Terminal 1

Publisher

Synthetic Mode

ros2 run signal_publisher publisher_node --mode synthetic

Replay Mode

ros2 run signal_publisher publisher_node --mode replay

Terminal 2

C++ Processor

ros2 run signal_processor_cpp signal_processor_cpp

Terminal 3

Python Processor

ros2 run signal_processor_python processor_node

Inspect topic data:

kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ cd ~/signal_pipeline_ws

source /opt/ros/jazzy/setup.bash

source install/setup.bash

ros2 topic hz /signal/float

average rate: 48.755

min: 0.008s max: 0.063s std dev: 0.00760s window: 51

average rate: 49.410

min: 0.008s max: 0.063s std dev: 0.00584s window: 102

average rate: 49.569

min: 0.008s max: 0.063s std dev: 0.00517s window: 152

average rate: 49.374

min: 0.008s max: 0.063s std dev: 0.00506s window: 201

average rate: 49.580

min: 0.005s max: 0.063s std dev: 0.00499s window: 252

Summary

The build and execution procedure confirms that the ROS 2 workspace compiles successfully, all nodes start correctly, topics are published and subscribed as expected, and the C++ and Python processing nodes receive and process the signal data using the shared DSP library.

CHAPTER 16

VALIDATION METHODOLOGY

16.1 Overview

Validation is an essential phase in software development that ensures the implemented system satisfies the functional requirements and behaves as expected under different operating conditions. In this project, validation was performed by executing the ROS 2 nodes, monitoring communication between publishers and subscribers, verifying the correctness of the Digital Signal Processing (DSP) algorithms, and confirming interoperability between the C++ and Python implementations.

The validation process included testing both the Synthetic Mode and Replay Mode of the publisher. The outputs from the C++ and Python processing nodes were observed and compared to verify that the pybind11 bindings correctly exposed the standalone C++ DSP library.

16.2 Test Environment

Component	Configuration
Operating System	Ubuntu 24.04 LTS
ROS Distribution	ROS 2 Jazzy
Programming Languages	C++17, Python 3
Build System	Colcon, CMake
Version Control	Git & GitHub
Data Source	Synthetic Signals and sensor_log.csv

16.3 Validation Tests

Test Case 1 – Build Verification

Objective: Ensure that the project compiles successfully.

Command

colcon build

OUTPUT:

```
kokkurohithkumar@kokkurohithkumar-VirtualBox:~$ cd ~/signal_pipeline_ws
kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ source /opt/r
os/jazzy/setup.bash
kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ source instal
l/setup.bash
kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ colcon build
--symlink-install
Starting >>> signal_pipeline
Starting >>> signal_pipeline_python_bindings
Starting >>> signal_processor_cpp
Starting >>> signal_processor_python
[0.849s] WARNING:colcon.colcon_cmake.task.cmake.build:Could not run installation
 step for package 'signal_pipeline' because it has no 'install' target
Finished <<< signal_pipeline [0.37s]
Starting >>> signal_publisher
Finished <<< signal_processor_cpp [0.66s]
Finished <<< signal_pipeline_python_bindings [0.68s]
Finished <<< signal_processor_python [2.24s]
Finished <<< signal_publisher [2.10s]

Summary: 5 packages finished [2.79s]
kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ SS
```

FIG: COLCON BUILD.

Test Case 2 – Publisher Execution

Objective: Verify that the Signal Publisher starts correctly and publishes data.

Command

ros2 run signal_publisher publisher_node --mode synthetic

```
kokkurohithkumar@kokkurohithkumar-VirtualBox:~$ cd signal_pipeline_ws
kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ source /opt/r
os/jazzy/setup.bash
kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ source instal
l/setup.bash
erminal ohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ ros2 run sign
l_publisher publisher_node --mode synthetic
[INFO] [1784992513.571340176] [signal_publisher]: Running in synthetic mode | Ji
tter=0.0s | Drop Rate=0.0
[INFO] [1784992513.579293104] [signal_publisher]: [Synthetic] Float=0.319  Int=3
1
[INFO] [1784992513.599090578] [signal_publisher]: [Synthetic] Float=0.622  Int=6
2
[INFO] [1784992513.618041909] [signal_publisher]: [Synthetic] Float=0.664  Int=6
6
[INFO] [1784992513.639318708] [signal_publisher]: [Synthetic] Float=0.715  Int=7
1
[INFO] [1784992513.657817505] [signal_publisher]: [Synthetic] Float=0.755  Int=7
5
[INFO] [1784992513.880240712] [signal_publisher]: [Synthetic] Float=0.257  Int=2
5
[INFO] [1784992513.896651789] [signal_publisher]: [Synthetic] Float=0.149  Int=1
4
[INFO] [1784992513.917586045] [signal_publisher]: [Synthetic] Float=0.271  Int=2
```

FIG: PUBLISHER_NODE SYNTHETIC.

Test Case 3 – Publisher Replay

Objective: Verify that the Signal Publisher starts correctly and publishes data.

Command

ros2 run signal_publisher publisher_node --mode replay.

FIG: publisher node replay.

Test Case 4 – C++ Processor

Objective: Verify that the C++ processing node receives and filters data.

Command

ros2 run signal_processor_cpp signal_processor_cpp

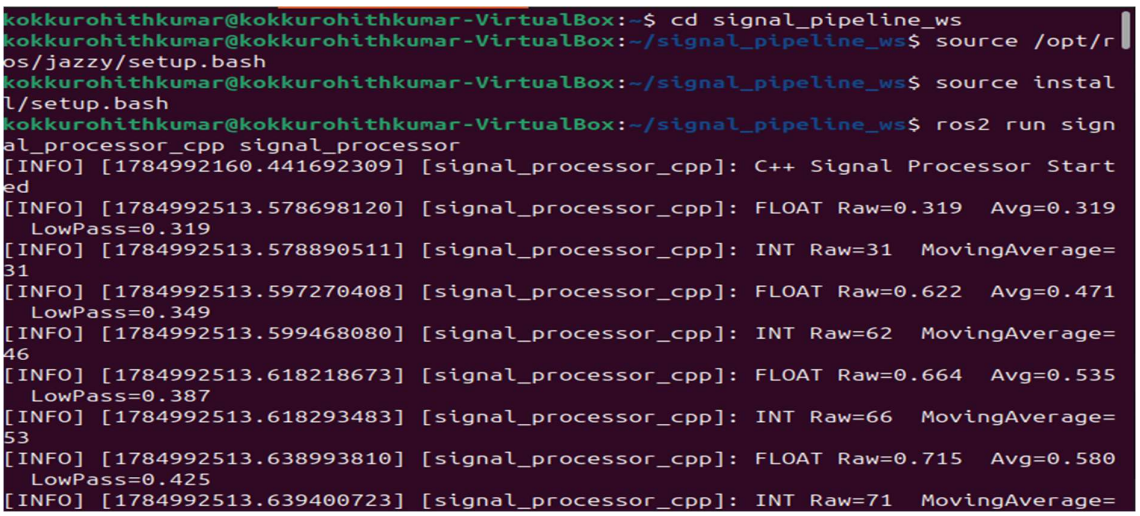


Fig: signal_processor output.

Test Case 5– Python Processor

Objective: Verify Python processing using pybind11.

Command

ros2 run signal_processor_python processor_node

```
kokkurohithkumar@kokkurohithkumar-VirtualBox:~$ cd signal_pipeline_ws
kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ source /opt/r
os/jazzy/setuo.bash
bash: /opt/ros/jazzy/setuo.bash: No such file or directory
kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ source /opt/r
os/jazzy/setup.bash
kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ source instal
l/setup.bash
kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ ros run signa
l_processor_python processor_node
Command 'ros' not found, but there are 13 similar ones.
/Box_GAs_7.2.0
kokkurohithkumar@kokkurohithkumar-VirtualBox:~/signal_pipeline_ws$ ros2 run sign
al_processor_python processor_node
[INFO] [1784992311.645810358] [signal_processor_python]: Python Signal Processor
 Started
[INFO] [1784992513.582195867] [signal_processor_python]: INT Raw=31  Avg=31
[INFO] [1784992513.583379254] [signal_processor_python]: FLOAT Raw=0.319  Avg=0.
319  LowPass=0.319
[INFO] [1784992513.598447576] [signal_processor_python]: FLOAT Raw=0.622  Avg=0.
471  LowPass=0.349
[INFO] [1784992513.600734957] [signal_processor_python]: INT Raw=62  Avg=46
[INFO] [1784992513.618109440] [signal_processor_python]: FLOAT Raw=0.664  Avg=0.
535  LowPass=0.387
[INFO] [1784992513.618848490] [signal_processor_python]: INT Raw=66  Avg=53
```

Fig: processor_node.py output.

Test Case 5 – ROS Topic Verification

Command

ros2 topic list

```
kokkurohithkumar@kokkurohithkumar-VirtualBox:~$ ros2 topic list
/parameter_events
/rosout
/signal/float
/signal/int
kokkurohithkumar@kokkurohithkumar-VirtualBox:~$
```

Fig: The output of topic list.

Test Case 6 – ROS Node Verification

Command

ros2 node list

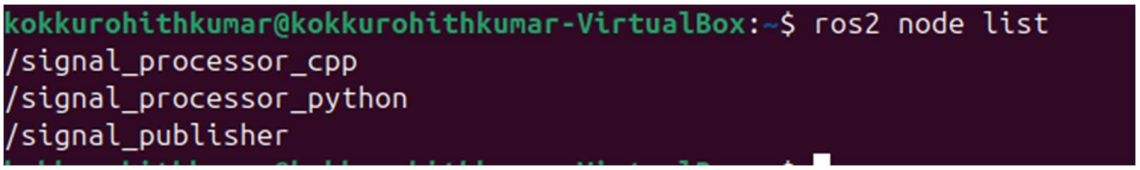


Fig: The output of node_list.

Test Case 7 – Topic Monitoring

Command

ros2 topic echo /signal/int

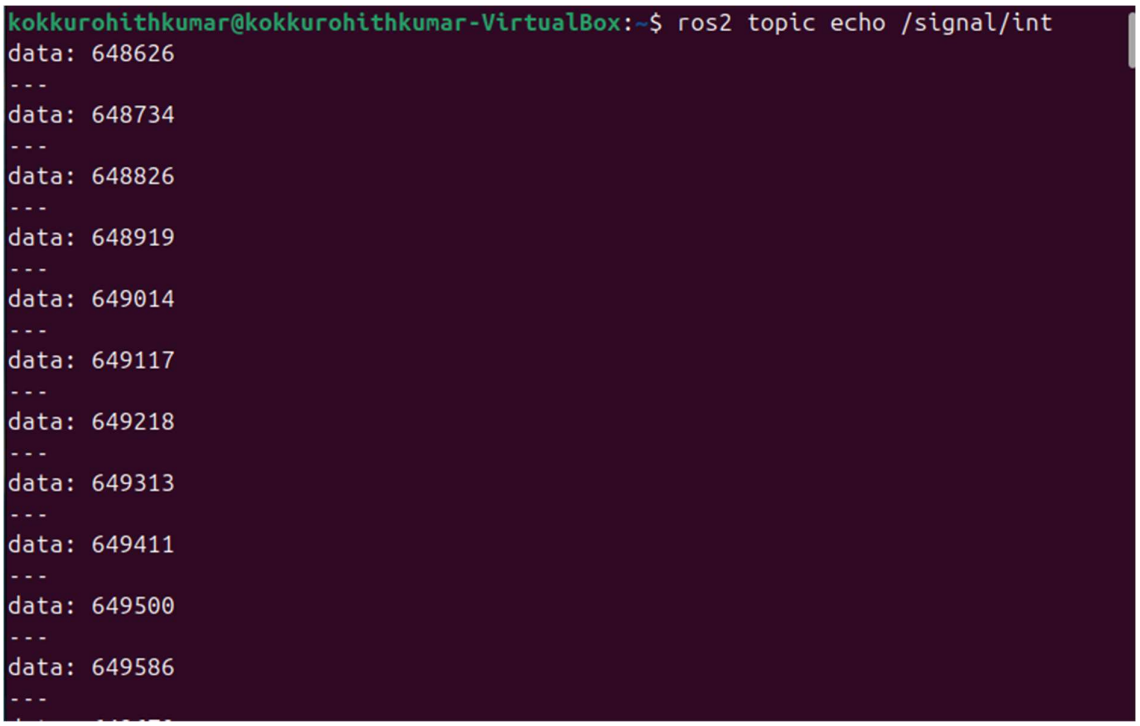


Fig: The output of echo_signal.

CHAPTER 17

CONCLUSION

The **ROS 2 Signal Processing Pipeline** project successfully demonstrates the design and implementation of a modular, reusable, and scalable signal processing framework using ROS 2, C++17, Python 3, and pybind11. A standalone C++ DSP library was developed to provide reusable implementations of common filtering algorithms, while Python bindings enabled the same functionality to be accessed from Python without code duplication.

The project validates the effectiveness of the ROS 2 Publisher–Subscriber communication model for real-time signal exchange and highlights the advantages of combining high-performance C++ processing with Python-based application development. The successful execution of both Synthetic and Replay modes, along with the verification of node communication and topic transmission, confirms that the implemented architecture satisfies the assessment objectives.

In addition to demonstrating technical proficiency in ROS 2, digital signal processing, and software engineering, the project emphasizes best practices such as modular design, version control using Git, build automation with CMake and Colcon, and comprehensive documentation. The resulting solution provides a strong foundation for future extensions involving physical sensors, advanced DSP algorithms, and autonomous robotic applications.